

BLOCO 5 • SISTEMAS OPERACIONAIS DISTRIBUÍDOS

# Sistemas Operacionais Distribuídos

## Gerência de Processos e Escalonamento

## Distribuído

Aula 25 • Sistemas Distribuídos • Uni-FACEF

Prof. Leandro Borges

# Agenda de hoje

**01** Da Aula 24 para a Aula 25: de arquitetura a processos

**02** Threads vs. processos em um sistema distribuído

**03** Migração de processos: motivações e desafios

**04** Escalonamento distribuído: decidir onde, não só quando

**05** Estratégias de balanceamento de carga

**06** Próxima aula e referências

# Da arquitetura ao processo: o que falta resolver

Na Aula 24 vimos os modelos de arquitetura de um Sistema Operacional Distribuído (SOD) — workstation, processor pool e híbrido — e como o middleware viabiliza a ilusão de máquina única na prática. Mas todos esses modelos compartilham um problema em aberto: uma vez que existem processos rodando em várias máquinas, quem decide onde cada processo executa e como ele se move entre nós? É exatamente esse problema — gerência de processos e escalonamento distribuído — que aprofundamos hoje.

1

## SO tradicional

Um único escalonador decide qual processo roda agora, numa única CPU/máquina.

2

## SOD (Aula 24)

Vários nós, middleware simula um sistema único — mas alguém ainda decide onde cada processo vive.

3

## Foco de hoje

Threads, migração de processos e escalonamento distribuído: o "onde", além do "quando".

# Threads vs. processos em um sistema distribuído

Um processo é uma unidade pesada: tem seu próprio espaço de endereçamento, e criá-lo ou trocar de contexto entre processos custa caro. Uma thread é uma unidade de execução leve dentro de um processo — várias threads de um mesmo processo compartilham memória e recursos abertos, mas têm sua própria pilha de execução. Em servidores distribuídos, que precisam atender muitas requisições concorrentes, threads permitem paralelismo sem o custo de criar um processo novo a cada cliente.

## Processo

- Espaço de endereçamento próprio e isolado
- Troca de contexto entre processos é cara
- Comunicação entre processos exige IPC (memória compartilhada, sockets, etc.)
- Criação é lenta — envolve alocar novo espaço de memória

## Thread

- Compartilha memória e recursos com as demais threads do processo
- Troca de contexto é muito mais barata
- Comunicação entre threads é direta, via memória compartilhada
- Criação é rápida — ideal para atender muitas requisições concorrentes

# Thread por requisição: concorrência em servidores

Um padrão comum em servidores distribuídos é o modelo de thread por requisição (thread-per-request): a cada nova requisição de um cliente, o servidor dispara uma thread dedicada para atendê-la, enquanto a thread principal continua livre para aceitar novas conexões. Como threads são leves, o servidor consegue atender muitos clientes simultaneamente sem pagar o custo de criar um processo completo para cada um.



# Migração de processos: mover a execução em andamento

Migração de processos é a capacidade de transferir um processo em execução de uma máquina para outra, preservando seu estado — como se ele nunca tivesse parado. Diferente de simplesmente iniciar uma nova cópia em outro nó, a migração carrega consigo o progresso já feito: valores de variáveis, pilha de execução e ponto exato de onde o processo estava.

## Por que migrar um processo?



### Balanceamento de carga

Mover processos de nós sobrecarregados para nós mais ociosos, distribuindo melhor o trabalho.



### Proximidade dos dados

Levar a computação para perto de onde os dados que ela usa estão armazenados, reduzindo tráfego de rede.



### Manutenção do sistema

Esvaziar uma máquina que vai passar por manutenção ou desligamento, sem interromper os processos.

# Migração de processos: os desafios técnicos

Migrar um processo é caro e complexo, porque é preciso reconstruir fielmente seu estado na máquina de destino, sem que o processo (ou quem se comunica com ele) perceba a mudança.

1

## Transferir a memória

Todo o espaço de endereçamento do processo (código, pilha, heap) precisa ser copiado para o nó de destino.

2

## Reconectar recursos abertos

Arquivos abertos, sockets de rede e outros descritores precisam continuar válidos após a mudança de máquina.

3

## Redirecionar mensagens em trânsito

Quem já enviava mensagens para o processo na máquina antiga precisa passar a alcançá-lo no novo endereço.

4

## Minimizar o tempo de congelamento

Quanto mais tempo o processo fica pausado durante a transferência, maior o impacto percebido pelo usuário.

# Escalonamento distribuído: o "onde", além do "quando"

Num SO tradicional, o escalonador só decide qual processo roda agora numa única CPU. Num sistema distribuído, existe uma pergunta adicional, anterior a essa: em qual máquina esse processo deveria rodar? Só depois disso é que o escalonador local daquele nó decide quando, entre seus processos, cada um executa.

### 1. Escalonamento distribuído (entre máquinas)

Decide em qual nó da rede o processo deve ser colocado para executar, considerando carga, localização dos dados e disponibilidade de recursos.



### 2. Escalonamento local (dentro da máquina)

O SO local daquele nó decide, entre todos os processos já alocados a ele, qual executa agora na CPU — o escalonamento clássico já conhecido.

*O escalonamento distribuído não substitui o escalonamento local — ele acontece antes, decidindo o destino do processo.*



# Estratégias de balanceamento: estático vs. dinâmico

Uma vez que existem vários nós candidatos, como decidir onde colocar cada processo? As estratégias de balanceamento de carga se dividem, primeiro, entre estáticas e dinâmicas.

## Balanceamento estático

- Decisão de alocação é fixa, definida antecipadamente (ex.: por regra ou round-robin)
- Não observa a carga real dos nós no momento da decisão
- Simples de implementar e com baixo overhead
- Pode desperdiçar recursos se a carga real mudar ao longo do tempo

## Balanceamento dinâmico

- Decisão é tomada com base na carga atual medida em cada nó
- Reage a mudanças de carga em tempo real
- Mais eficiente na prática, mas exige coletar e trocar informações de carga
- Introduz overhead de monitoramento e comunicação entre nós

# Quem inicia o balanceamento: sender vs. receiver

Dentro do balanceamento dinâmico, uma segunda escolha de projeto é: quem toma a iniciativa de agir quando a carga está desbalanceada?



## Sender-initiated

*iniciado pelo remetente*

A máquina sobrecarregada é quem toma a iniciativa: percebe que está com carga alta e procura ativamente por um nó mais ocioso para transferir trabalho.

**Funciona bem quando a carga geral do sistema é baixa a moderada.**



## Receiver-initiated

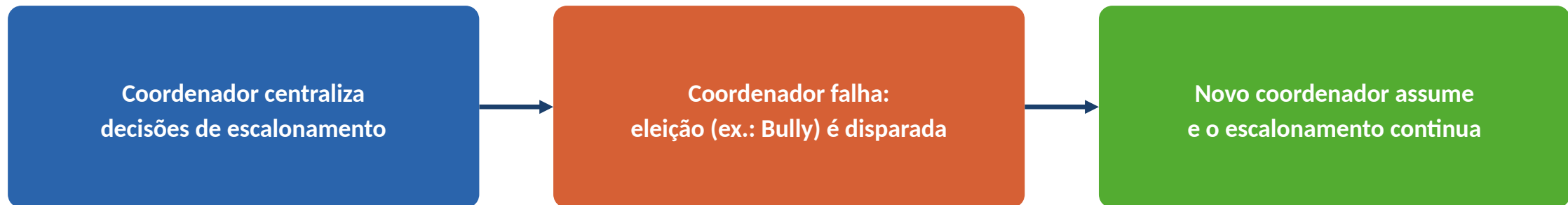
*iniciado pelo destinatário*

A máquina ociosa é quem toma a iniciativa: percebe que tem capacidade livre e anuncia ou solicita ativamente trabalho de outros nós.

**Funciona bem quando a carga geral do sistema já é alta.**

## Coordenando a decisão: o papel dos algoritmos de eleição

Algumas estratégias de escalonamento distribuído dependem de um nó coordenador para centralizar decisões — por exemplo, para reunir informações de carga de todos os nós antes de decidir para onde migrar um processo. Quando esse coordenador falha ou precisa ser trocado, entra em cena o problema de eleição que já estudamos: algoritmos como o Bully permitem que os nós escolham um novo coordenador de forma consistente, sem intervenção manual, mantendo o escalonamento distribuído funcionando mesmo diante de falhas.



*Já vimos o algoritmo Bully em detalhe nas aulas de coordenação e consenso — aqui ele reaparece apenas como mecanismo de apoio ao escalonamento distribuído.*

PRÓXIMA AULA

# Aula 26 — Sistemas Operacionais Distribuídos: Sistemas de Arquivos Distribuídos

Depois de vermos como processos são gerenciados e escalonados entre máquinas, vamos ver como um SOD lida com o armazenamento: como arquivos são distribuídos, replicados e acessados de forma transparente entre diferentes nós.

# Referências

TANENBAUM, A. S.; VAN STEEN, M. Distributed Systems: Principles and Paradigms. 3. ed. Pearson, 2017. Cap. 3 (Processos) — threads, migração de código e algoritmos de escalonamento.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. Distributed Systems: Concepts and Design. 5. ed. Addison-Wesley, 2011. Cap. 4 (Interação) e Cap. 6 (Coordenação e Acordo) — modelos de processo e coordenação distribuída.

TANENBAUM, A. S.; VAN STEEN, M. Distributed Systems. 4. ed. 2023 (edição digital gratuita) — capítulo sobre processos, threads e migração.